

## 22. Bölüm

# Birim Testi ve Test Güdümlü Geliştirme

Yazılım geliştirmenin en kritik ama çoğu zaman ihmal edilen boyutu **test**tir. “Kod çalışıyor” demek, kodun *doğru* çalıştığı anlamına gelmez. Kurumsal yazılımlarda bir geliştirici yazdığı kodu aylar sonra tekrar ele alır, ekip arkadaşı aynı modülü değiştirir veya yeni bir gereksinim nedeniyle mevcut davranış güncellenir. Bu değişikliklerin sistemi bozmadığını güvence altına alan tek mekanizma **test**tir.

**Birim testi** (unit test), bir yazılımın en küçük bağımsız parçasını — bir yöntemi, bir sınıfı — izole biçimde doğrular. Birim testleri milisaniyeler içinde çalışır, sürekli entegrasyon (CI) boru hatlarında otomatik yürütülür ve geliştiriciyi anında bilgilendirir.

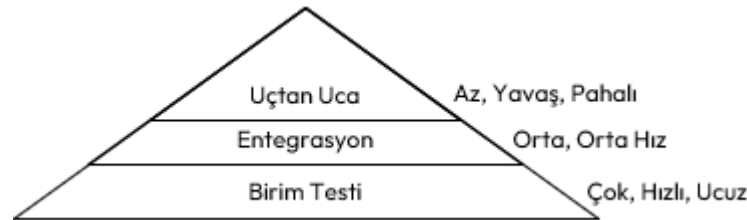
Bölümün sonunda aşağıdaki soruları rahatlıkla yanıtlayabileceksiniz:

- ◇ Birim, entegrasyon ve uçtan uca test arasındaki fark nedir?
- ◇ xUnit.net ile nasıl test yazılır, nasıl çalıştırılır?
- ◇ NUnit’in xUnit’ten farkı nedir, hangi durumda hangisi tercih edilir?
- ◇ Moq ile bağımlılıklar nasıl taklit edilir (mock)?
- ◇ Test Güdümlü Geliştirme (**Test Driven Development**) döngüsü nasıl işler?
- ◇ Temiz ve bakımı kolay test nasıl yazılır?
- ◇ Eş zamansız kodlar nasıl test edilir?

## 22.1 Test Piramidi ve Test Türleri

### §22.1.1 Test Piramidi

Modern yazılım geliştirmede testler üç katmana ayrılır. Bu yapı **test piramidi** olarak adlandırılır ve her katmanın farklı bir amacı, farklı bir maliyeti vardır.



Şekil 22.1: Test Piramidi

| Tür         | Kapsam              | Hız       | Adet | Örnek                 |
|-------------|---------------------|-----------|------|-----------------------|
| Birim       | Tek sınıf/yöntem    | Çok hızlı | Çok  | Hesapla() doğrular    |
| Entegrasyon | Birden fazla katman | Orta      | Orta | Servis + DB birlikte  |
| Uçtan Uca   | Tüm sistem          | Yavaş     | Az   | Tarayıcı ile tam akış |

Tablo 22.1: Test Türleri Karşılaştırması

### §22.1.2 İyi Bir Birim Testin Özellikleri — FIRST

Kaliteli birim testleri **FIRST** ilkesine uyar:

- ◇ **Fast** (Hızlı): Milisaniyeler içinde biter. Yüzlerce test saniyeler içinde çalışır.
- ◇ **Isolated** (İzole): Başka testlere, ağa, dosyaya veya veri tabanına bağımlı değildir.
- ◇ **Repeatable** (Tekrarlanabilir): Her çalıştırmada aynı sonucu verir.
- ◇ **Self-validating** (Kendini Doğrulayan): Geçti/kaldı sonucunu otomatik üretir, insan yorumuna ihtiyaç duymaz.
- ◇ **Timely** (Zamanında): Üretim kodu yazılmadan önce veya hemen ardından yazılır.

## 22.2 xUnit.net ile Birim Testi

**xUnit.net**, .NET ekosisteminin en yaygın kullanılan modern test çerçevesidir. Microsoft’un kendi projeleri, ASP.NET Core ve Entity Framework Core da xUnit ile test edilmektedir.

### §22.2.1 Proje Kurulumu

Test projesi oluşturmak için komut satırında şu komutlar çalıştırılır:

```
// Yeni bir xUnit test projesi oluştur
// dotnet new xunit -n HesapMakinesi.Testler

// Ana projeye referans ekle
// dotnet add HesapMakinesi.Testler reference HesapMakinesi

// NuGet paketleri (xUnit projesiyle otomatik gelir):
// xunit - test çerçevesi
// xunit.runner.visualstudio - Visual Studio entegrasyonu
// Microsoft.NET.Test.Sdk - .NET test SDK'sı
```

### §22.2.2 Test Edilecek Sınıf

Önce test yazılacak üretim kodunu tanımlayalım. Bu bölüm boyunca bu sınıfı kullanacağız.

```
// HesapMakinesi.cs - Ana proje
namespace HesapMakinesi;

public class HesapMakinesi
{
    public double Topla(double a, double b) => a + b;

    public double Çıkar(double a, double b) => a - b;

    public double Çarp(double a, double b) => a * b;

    public double Böl(double a, double b)
    {
        if (b == 0)
            throw new DivideByZeroException("Sıfıra bölme hatası.");
        return a / b;
    }

    public bool AsalMı(int sayı)
    {
        if (sayı < 2) return false;
        for (int i = 2; i <= Math.Sqrt(sayı); i++)
            if (sayı % i == 0) return false;
        return true;
    }
}
```

### §22.2.3 İlk xUnit Testi — [Fact]

[Fact], parametre almayan, tek bir senaryoyu doğrulayan test yöntemini işaretler.

```
// HesapMakinesiTestleri.cs - Test projesi
using Xunit;

namespace HesapMakinesi.Testler;

public class HesapMakinesiTestleri
{
    // Test sınıfı her test için yeniden örnek oluşturur (xUnit davranışı)
    private readonly HesapMakinesi _hesap = new();

    [Fact]
    public void Topla_İkiPositifSayı_DoğruSonuçDöndürür()
    {
        // Arrange (Hazırla) - test girdileri ve beklenen değer
        double a = 3;
        double b = 4;
```

```

    double beklenen = 7;

    // Act (Uygula) – test edilecek yöntemi çağır
    double gerçek = _hesap.Topla(a, b);

    // Assert (Doğrula) – beklenen ile gerçeği karşılaştır
    Assert.Equal(beklenen, gerçek);
}

[Fact]
public void Topla_NegatifSayılar_DogruSonucDondurur()
{
    double sonuc = _hesap.Topla(-5, -3);
    Assert.Equal(-8, sonuc);
}

[Fact]
public void Bol_SifiraBolme_IstisnaDondurur()
{
    // İstisna fırlatılmasını doğrula
    Assert.Throws<DivideByZeroException>(() =>
        _hesap.Böl(10, 0));
}

[Fact]
public void Bol_NormalBolme_DogruSonucDondurur()
{
    double sonuc = _hesap.Böl(10, 4);
    // Kayan nokta karşılaştırmasında hassasiyet belirt
    Assert.Equal(2.5, sonuc, precision: 5);
}
}

```

### §22.2.4 Test İsimlendirme Kuralı

Test adı üç bölümden oluşmalıdır:

[Yöntem]\_[Senaryo]\_[BeklenenSonuç]

| Kötü İsim      | İyi İsim                                  |
|----------------|-------------------------------------------|
| Test1()        | Topla_İkiPozitif_DogruToplamar()          |
| BölmeTest()    | Böl_SifirPaydayla_DivideByZeroFirlatir()  |
| AşırıYükTest() | AsalMı_BilinçAşağısıSayı_YanlışDondurur() |

Tablo 22.2: Test İsimlendirme Örnekleri

### §22.2.5 Parametrel Test — [Theory] ve [InlineData]

Aynı mantığı farklı girdilerle doğrulamak için [Theory] ve [InlineData] kullanılır. Bu yapı test tekrarını ortadan kaldırır.

```

public class HesapMakinesiTheoryTestleri
{
    private readonly HesapMakinesi _hesap = new();

    // [Theory] – aynı test, farklı girdilerle çalışır
    [Theory]
    [InlineData(3, 4, 7)] // 3 + 4 = 7
    [InlineData(-1, -2, -3)] // -1 + -2 = -3
    [InlineData(0, 0, 0)] // 0 + 0 = 0
    [InlineData(100, -50, 50)] // 100 + (-50) = 50
    public void Topla_CesitliGirdiler_DogruSonucDondurur(
        double a, double b, double beklenen)
    {
        double gerçek = _hesap.Topla(a, b);
    }
}

```

```

        Assert.Equal(beklenen, gerçek);
    }

    [Theory]
    [InlineData(2, true)] // 2 asal
    [InlineData(3, true)] // 3 asal
    [InlineData(4, false)] // 4 asal değil
    [InlineData(17, true)] // 17 asal
    [InlineData(1, false)] // 1 asal değil
    [InlineData(0, false)] // 0 asal değil
    public void AsalMı_ÇeşitliSayılar_DoğruSonuçDöndürür(int sayı, bool beklenen)
    {
        bool gerçek = _hesap.AsalMı(sayı);
        Assert.Equal(beklenen, gerçek);
    }
}

```

### §22.2.6 [MemberData] ile Karmaşık Veri Setleri

Karmaşık nesneler veya büyük veri setleri için [MemberData] kullanılır.

```

public class BölmeTestleri
{
    private readonly HesapMakinesi _hesap = new();

    // Statik özellik olarak test verisi tanımla
    public static IEnumerable<object[]> BölmeVerisi =>
        new List<object[]>
        {
            new object[] { 10.0, 2.0, 5.0 },
            new object[] { 9.0, 3.0, 3.0 },
            new object[] { 7.0, 2.0, 3.5 },
            new object[] { -8.0, 4.0, -2.0 },
        };

    [Theory]
    [MemberData(nameof(BölmeVerisi))]
    public void Böl_GeçerliGirdiler_DoğruSonuçDöndürür(
        double bölünen, double bölen, double beklenen)
    {
        double gerçek = _hesap.Böl(bölünen, bölen);
        Assert.Equal(beklenen, gerçek, precision: 10);
    }
}

```

### §22.2.7 xUnit Doğrulama Yöntemleri

```

// Eşitlik doğrulama
Assert.Equal(beklenen, gerçek);
Assert.NotEqual(beklenmeyen, gerçek);

// Boolean doğrulama
Assert.True(koşul);
Assert.False(koşul);

// Null doğrulama
Assert.Null(nesne);
Assert.NotNull(nesne);

// İstisna doğrulama
Assert.Throws<ArgumentException>(() => Yöntem());
var hata = Assert.Throws<InvalidOperationException>(() => Yöntem());
Assert.Contains("beklenen mesaj", hata.Message);

```

```
// Koleksiyon doęrulama
Assert.Empty(koleksiyon);
Assert.NotEmpty(koleksiyon);
Assert.Single(koleksiyon);
Assert.Contains(eleman, koleksiyon);

// T r doęrulama
Assert.IsType<HesapMakinesi>(nesne);
Assert.IsAssignableFrom<IHesapMakinesi>(nesne);

// Yaklařık eřitlik (kayan nokta)
Assert.Equal(3.14, sonu, precision: 2); // 2 ondalık basamak
```

## ř22.2.8 Test Yařam D ng s  — Oluřturucu ve IDisposable

xUnit'te her test y ntemi iin sınıf yeniden oluřturulur. Paylařılan kaynaklar `IClassFixture` ile y netilir.

```
using Xunit;
using Xunit.Abstractions;

// ITestOutputHelper – test ıktısını yakala
public class YařamD ng s Testleri : IDisposable
{
    private readonly ITestOutputHelper _ıktı;
    private readonly HesapMakinesi _hesap;

    // Her test iin alıřır – kurulum
    public YařamD ng s Testleri(ITestOutputHelper ıktı)
    {
        _ıktı = ıktı;
        _hesap = new HesapMakinesi();
        _ıktı.WriteLine("Test bařladı.");
    }

    [Fact]
    public void Topla_ rnek_alıřıyor()
    {
        double sonu = _hesap.Topla(2, 3);
        _ıktı.WriteLine($"Sonu: {sonu}");
        Assert.Equal(5, sonu);
    }

    // Her test sonrası alıřır – temizlik
    public void Dispose()
    {
        _ıktı.WriteLine("Test bitti.");
    }
}

// Paylařılan pahalı kaynak iin Fixture
public class VeritabanıFixture : IDisposable
{
    public string BaęlantıSatırı { get; } = "Server=localhost;Database=Test";

    public VeritabanıFixture()
    {
        // Veri tabanı baęlantısını bir kez kur (t m testler iin)
        Console.WriteLine("Veri tabanı baęlantısı kuruldu.");
    }

    public void Dispose()
    {
        Console.WriteLine("Veri tabanı baęlantısı kapatıldı.");
    }
}
```

```

    }
}

// IClassFixture - tüm testler için fixture'ı paylaş
public class VeritabanıTestleri : IClassFixture<VeritabanıFixture>
{
    private readonly VeritabanıFixture _fixture;

    public VeritabanıTestleri(VeritabanıFixture fixture)
    {
        _fixture = fixture; // Tek örnek, tüm testler paylaşır
    }

    [Fact]
    public void BağlantıSatırı_Boş_Değil()
    {
        Assert.NotEmpty(_fixture.BağlantıSatırı);
    }
}

```

## 22.3 NUnit ile Birim Testi

**NUnit**, .NET dünyasının köklü test çerçevelerinden biridir. JUnit'ten esinlenerek tasarlanmıştır ve özellikle .NET Framework döneminden gelen projelerde yaygın kullanılmaktadır.

### §22.3.1 NUnit Kurulumu

```

// dotnet new nunit -n HesapMakinesi.NUnitTestler
// Paketler:
// NUnit
// NUnit3TestAdapter
// Microsoft.NET.Test.Sdk

```

### §22.3.2 Temel NUnit Testi

```

using NUnit.Framework;

namespace HesapMakinesi.NUnitTestler;

[TestFixture] // NUnit'te test sınıfını işaretler
public class HesapMakinesiNUnitTestleri
{
    private HesapMakinesi _hesap = null!;

    [SetUp] // Her testten önce çalışır (xUnit'te oluşturu)
    public void Kurulum()
    {
        _hesap = new HesapMakinesi();
    }

    [TearDown] // Her testten sonra çalışır (xUnit'te IDisposable)
    public void Temizlik()
    {
        // Kaynakları serbest bırak
    }

    [OneTimeSetUp] // Tüm testlerden önce bir kez (xUnit'te IClassFixture)
    public void SınıfKurulumu()
    {
        Console.WriteLine("Test sınıfı başlatıldı.");
    }

    [Test] // [Fact] karşılığı
    public void Topla_İkiPozitifSayı_DoğruSonuç()

```

```

{
    double sonu = _hesap.Topla(3, 4);
    Assert.That(sonu, Is.EqualTo(7)); // NUnit'in akıcı s zdizimi
}

[Test]
public void B l_SıfıraB lme_İstisnaD nd r r()
{
    Assert.Throws<DivideByZeroException>(() => _hesap.B l(10, 0));
}

[TestCase(3, 4, 7)] // [InlineData] karřılıęı
[TestCase(-1, -2, -3)]
[TestCase(0, 0, 0)]
public void Topla_eřitliGirdiler_DoęruSonu(double a, double b, double beklenen)
{
    double gerek = _hesap.Topla(a, b);
    Assert.That(gerek, Is.EqualTo(beklenen));
}
}

```

###  22.3.3 NUnit Akıcı Doęrulama — Assert.That

NUnit'in `Assert.That` s zdizimi son derece okunabilirdir:

```

// Eřitlik
Assert.That(gerek, Is.EqualTo(beklenen));
Assert.That(gerek, Is.Not.EqualTo(beklenmeyen));

// B y kl k karřılařtırma
Assert.That(sayı, Is.GreaterThan(0));
Assert.That(sayı, Is.LessThanOrEqualTo(100));
Assert.That(sayı, Is.InRange(1, 10));

// Null kontrol 
Assert.That(nesne, Is.Null);
Assert.That(nesne, Is.Not.Null);

// String doęrulama
Assert.That(metin, Does.StartWith("Merhaba"));
Assert.That(metin, Does.Contain("d nya"));
Assert.That(metin, Does.Match(@"\d+")); // Regex

// Koleksiyon doęrulama
Assert.That(liste, Is.Empty);
Assert.That(liste, Has.Count.EqualTo(3));
Assert.That(liste, Does.Contain(42));
Assert.That(liste, Is.Ordered);
Assert.That(liste, Is.Unique);

// T r doęrulama
Assert.That(nesne, Is.InstanceOf<HesapMakinesi>());

```

###  22.3.4 xUnit ve NUnit Karřılařtırması

Karřılařtırma tablosu Tablo 22.3'de verilmiřtir.

## 22.4 Moq ile Baęımlılık Taklidi

Gerek d nya sınıfları nadiren tek bařına alıřır. Bir servis veri tabanına eriřir, bir API'ye istek atar veya dosya sistemini kullanır. Birim testlerinde bu *dıř baęımlılıklar* gerek sistemlere eriřmeden taklit edilmelidir. Bu iřleme **mocking** (taklitilik) denir.

**Moq**, .NET iin en yaygın kullanılan taklitilik k t phanesidir.

| Özellik                   | xUnit.net             | NUnit                 |
|---------------------------|-----------------------|-----------------------|
| Test işaretleyici         | [Fact]                | [Test]                |
| Parametreliliği           | [Theory]/[InlineData] | [TestCase]            |
| Sınıf işaretleyici        | Gerek yok             | [TestFixture]         |
| Her test öncesi kurulum   | Oluşturucu            | [SetUp]               |
| Her test sonrası temizlik | IDisposable           | [TearDown]            |
| Paylaşılan kaynak         | IClassFixture         | [OneTimeSetUp]        |
| Doğrulama stili           | Assert.Equal(...)     | Assert.That(Is...)    |
| .NET tavsiyesi            | Microsoft tercih eder | Köklü, geniş topluluk |

Tablo 22.3: xUnit.net ve NUnit Karşılaştırması

### §22.4.1 Taklit İçin Arayüz Tasarımı

Moq yalnızca arayüz veya soyut sınıf üzerinde çalışır. Bu da kodunuzu **gevşek bağlı** (**loosely coupled**) tasarlamaya zorlar — ki bu zaten iyi bir alışkanlıktır.

```
// Üretim kodu – arayüzler üzerinden bağımlılık
namespace Kütüphane;

// Kullanıcı veri erişim arayüzü
public interface IKullanıcıDepo
{
    Kullanıcı? BulById(int id);
    void Kaydet(Kullanıcı kullanıcı);
    bool EmailMevcutMu(string email);
}

// E-posta servis arayüzü
public interface IEpostaServisi
{
    void HoşgeldinGönder(string email, string ad);
}

// Kullanıcı modeli
public record Kullanıcı(int Id, string Ad, string Email, bool Aktif);

// Test edilecek servis – bağımlılıklar arayüz üzerinden enjekte edilir
public class KullanıcıServisi
{
    private readonly IKullanıcıDepo _depo;
    private readonly IEpostaServisi _epostaServisi;

    public KullanıcıServisi(IKullanıcıDepo depo, IEpostaServisi epostaServisi)
    {
        _depo = depo;
        _epostaServisi = epostaServisi;
    }

    public Kullanıcı? BulById(int id)
    {
        if (id <= 0)
            throw new ArgumentException("Id sıfırdan büyük olmalıdır.", nameof(id));
        return _depo.BulById(id);
    }

    public void KaydetVeHoşgeldinGönder(Kullanıcı kullanıcı)
    {
        if (_depo.EmailMevcutMu(kullanıcı.Email))
            throw new InvalidOperationException($"'{kullanıcı.Email}' zaten kayıtlı.");

        _depo.Kaydet(kullanıcı);
        _epostaServisi.HoşgeldinGönder(kullanıcı.Email, kullanıcı.Ad);
    }
}
```



```

    }
}

```

## §22.4.2 Moq Kurulumu ve Temel Kullanım

```

// dotnet add package Moq
using Moq;
using Xunit;

namespace K t phane.Testler;

public class KullanıcıServisTestleri
{
    // Mock nesneleri sınıf d zeyinde tanımla
    private readonly Mock<IKullanıcıDepo> _depoMock;
    private readonly Mock<IEpostaServisi> _epostaMock;
    private readonly KullanıcıServisi _servis;

    public KullanıcıServisTestleri()
    {
        _depoMock = new Mock<IKullanıcıDepo>();
        _epostaMock = new Mock<IEpostaServisi>();

        // Mock nesnelerin .Object  zelliđi ger ek nesne gibi davranır
        _servis = new KullanıcıServisi(
            _depoMock.Object,
            _epostaMock.Object);
    }

    [Fact]
    public void BulById_MevcutId_KullanıcıD nd r r()
    {
        // Arrange – Mock davranışını ayarla
        Kullanıcı beklenen = new(1, "İlhan", "ilhan@ornek.com", true);

        // BulById(1)  ađrıldığında beklenen kullanıcıyı d nd r
        _depoMock
            .Setup(d => d.BulById(1))
            .Returns(beklenen);

        // Act
        Kullanıcı? ger ek = _servis.BulById(1);

        // Assert
        Assert.NotNull(ger ek);
        Assert.Equal("İlhan", ger ek.Ad);
        Assert.Equal("ilhan@ornek.com", ger ek.Email);
    }

    [Fact]
    public void BulById_SıfırId_ArgumentExceptionFırlatır()
    {
        // Id <= 0 i in istisna fırlatılmalı
        Assert.Throws<ArgumentException>(() => _servis.BulById(0));

        // Depo hi   ađrılmamalı (zaten ulaşamaz)
        _depoMock.Verify(d => d.BulById(It.IsAny<int>()), Times.Never);
    }

    [Fact]
    public void BulById_MevcutOlmayanId_NullD nd r r()
    {
        // BulById(99)  ađrıldığında null d nd r
    }
}

```

```

        _depoMock
        .Setup(d => d.BulById(99))
        .Returns((Kullanıcı?)null);

        Kullanıcı? gerçek = _servis.BulById(99);

        Assert.Null(gerçek);
    }
}

```

### §22.4.3 Yöntem Çağrılarını Doğrulama — Verify

**Verify** ile bir yöntemin kaç kez ve hangi parametrelerle çağrıldığı doğrulanır.

```

public class KaydetveHoşgeldinTestleri
{
    private readonly Mock<IKullanıcıDepo> _depoMock = new();
    private readonly Mock<IEpostaServisi> _epostaMock = new();
    private readonly KullanıcıServisi _servis;

    public KaydetveHoşgeldinTestleri()
    {
        _servis = new KullanıcıServisi(_depoMock.Object, _epostaMock.Object);
    }

    [Fact]
    public void KaydetVeHoşgeldinGönder_YeniKullanıcı_KaydedilirVeEmailGönderilir()
    {
        // Arrange
        Kullanıcı kullanıcı = new(0, "Ayşe", "ayse@ornek.com", true);

        _depoMock
        .Setup(d => d.EmailMevcutMu("ayse@ornek.com"))
        .Returns(false); // Email kayıtlı değil

        // Act
        _servis.KaydetVeHoşgeldinGönder(kullanıcı);

        // Assert – Kaydet tam olarak bir kez çağrıldı
        _depoMock.Verify(
            d => d.Kaydet(It.Is<Kullanıcı>(k => k.Email == "ayse@ornek.com")),
            Times.Once);

        // Assert – Hoşgeldin e-postası gönderildi
        _epostaMock.Verify(
            e => e.HoşgeldinGönder("ayse@ornek.com", "Ayşe"),
            Times.Once);
    }

    [Fact]
    public void KaydetVeHoşgeldinGönder_MevcutEmail_İstisnaDöndürür()
    {
        // Arrange
        Kullanıcı kullanıcı = new(0, "Mehmet", "mevcut@ornek.com", true);

        _depoMock
        .Setup(d => d.EmailMevcutMu("mevcut@ornek.com"))
        .Returns(true); // Email zaten kayıtlı

        // Act & Assert
        Assert.Throws<InvalidOperationException>(() =>
            _servis.KaydetVeHoşgeldinGönder(kullanıcı));
    }
}

```

```
// Kaydet asla çağrılmamalı
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.Never);

// Hoşgeldin e-postası asla gönderilmemeli
_epostaMock.Verify(
e => e.HoşgeldinGönder(It.IsAny<string>(), It.IsAny<string>()),
Times.Never);
}
}
```

## §22.4.4 Moq — Gelişmiş Kullanım

```
// It.IsAny<T>() – herhangi bir değer
_depoMock.Setup(d => d.BulById(It.IsAny<int>())).Returns(kullanıcı);

// It.Is<T>(pred) – koşul filtresi
_depoMock.Setup(d => d.BulById(It.Is<int>(id => id > 0))).Returns(kullanıcı);

// Sıralı dönüşler – her çağrıda farklı değer
_depoMock
.SetupSequence(d => d.BulById(1))
.Returns(kullanıcı1) // 1. çağrı
.Returns(kullanıcı2) // 2. çağrı
.Returns((Kullanıcı?)null); // 3. çağrı

// Callback – çağrıldığında yan etki üret
_depoMock
.Setup(d => d.Kaydet(It.IsAny<Kullanıcı>()))
.Callback<Kullanıcı>(k => Console.WriteLine($"Kaydedildi: {k.Ad}"));

// İstisna fırlat
_depoMock
.Setup(d => d.BulById(-1))
.Throws<ArgumentException>();

// Özellik taklit etme
_depoMock
.SetupProperty(d => d.ToplamKayıt, 42);

// Times ile çağrı sayısı doğrulama
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.Once);
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.Exactly(3));
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.AtLeastOnce);
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.Never);
```

## 22.5 Eş Zamansız Kod Testi

Bölüm 20’da öğrendiğimiz `async/await` yapıları test edilirken özel dikkat gerektirir. xUnit ve NUnit her ikisi de `async Task` test yöntemlerini destekler.

### §22.5.1 Eş Zamansız Üretim Kodu

```
// Eş zamansız servis
public interface IHaberServisi
{
    Task<IEnumerable<Haber>> HaberleriGetirAsync(CancellationTokent iptal = default);
    Task<Haber?> HaberGetirAsync(int id);
}

public record Haber(int Id, string Başlık, DateTime YayınTarihi);

public class HaberÖzetleyici
{
    private readonly IHaberServisi _servis;
```

```

public HaberÖzetleyici(IHaberServisi servis)
{
    _servis = servis;
}

public async Task<string[]> BaşlıklarıGetirAsync()
{
    var haberler = await _servis.HaberleriGetirAsync();
    return haberler.Select(h => h.Başlık).ToArray();
}

public async Task<Haber> ZorunluHaberGetirAsync(int id)
{
    Haber? haber = await _servis.HaberGetirAsync(id);
    return haber ?? throw new KeyNotFoundException($"Haber bulunamadı: {id}");
}
}

```

## §22.5.2 Eş Zamansız Test Yöntemleri

```

using Moq;
using Xunit;

public class HaberÖzetleyiciTestleri
{
    private readonly Mock<IHaberServisi> _servisMock = new();
    private readonly HaberÖzetleyici _özetleyici;

    public HaberÖzetleyiciTestleri()
    {
        _özetleyici = new HaberÖzetleyici(_servisMock.Object);
    }

    [Fact]
    public async Task BaşlıklarıGetirAsync_ÜçHaber_ÜçBaşlıkDöndürür()
    {
        // Arrange – async döndüren mock
        List<Haber> haberler = new()
        {
            new(1, "C# 13 Yayınlandı", DateTime.Today),
            new(2, "AI Çağında Yazılım", DateTime.Today.AddDays(-1)),
            new(3, ".NET 10 Duyurusu", DateTime.Today.AddDays(-2)),
        };

        _servisMock
            .Setup(s => s.HaberleriGetirAsync(It.IsAny<CancellationToken>()))
            .ReturnsAsync(haberler); // ReturnsAsync – Task döndürür

        // Act – await ile eş zamansız çağır
        string[] başlıklar = await _özetleyici.BaşlıklarıGetirAsync();

        // Assert
        Assert.Equal(3, başlıklar.Length);
        Assert.Contains("C# 13 Yayınlandı", başlıklar);
    }

    [Fact]
    public async Task ZorunluHaberGetirAsync_MevcutOlmayan_KeyNotFoundExceptionFırlatır()
    {
        // Arrange
        _servisMock
            .Setup(s => s.HaberGetirAsync(999))

```

```

.ReturnsAsync((Haber?)null);

// Act & Assert - async istisnayı yakala
await Assert.ThrowsAsync<KeyNotFoundException>(async () =>
await _özetleyici.ZorunluHaberGetirAsync(999));
}

[Fact]
public async Task BaşlıklarıGetirAsync_BoşListe_BoşDiziDöndürür()
{
    _servisMock
    .Setup(s => s.HaberleriGetirAsync(It.IsAny<CancellationTokens>()))
    .ReturnsAsync(Enumerable.Empty<Haber>());

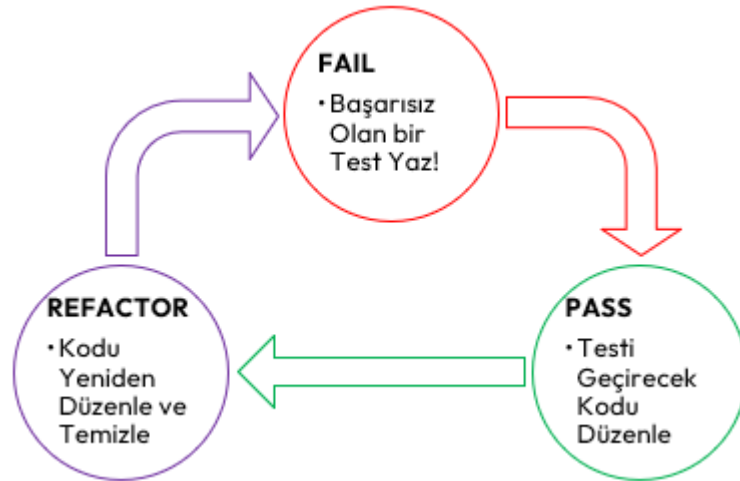
    string[] başlıklar = await _özetleyici.BaşlıklarıGetirAsync();

    Assert.Empty(başlıklar);
}
}

```

## 22.6 Test GÜdümlü Geliştirme — TDD

**Test GÜdümlü Geliştirme** (Test-Driven Development, TDD), kodu yazmadan önce test yazılan bir geliştirme yaklaşımıdır. Üç adımlı döngüsü **Kırmızı**→**Yeşil**→**Yeniden Düzenle** olarak özetlenir.



Şekil 22.2: TDD Döngüsü: Kırmızı → Yeşil → Yeniden Düzenle

### §22.6.1 TDD ile Alışveriş Sepeti Geliştirme

Aşağıda TDD yaklaşımıyla adım adım bir alışveriş sepeti sınıfı geliştirilmektedir.

#### Adım 1 — Kırmızı: İlk Testi Yaz (Kod Yok)

```

// 1. Test yaz - AlışverişSepeti sınıfı henüz YOK
// Bu test derlenmez bile - bu normal!
[Fact]
public void Ekle_TekÜrün_ToplamDoğru()
{
    // Bu aşamada AlışverişSepeti sınıfı yok - derleme hatası
    var sepet = new AlışverişSepeti();
    sepet.Ekle("Kalem", fiyat: 5.0m, adet: 2);

    Assert.Equal(10.0m, sepet.Toplam);
}

```

#### Adım 2 — Yeşil: Testi Geçirecek En Az Kodu Yaz

```

// Testi geçirecek minimum üretim kodu
public class AlışverişSepeti

```

```

{
    private decimal _toplam = 0;

    public decimal Toplam => _toplam;

    public void Ekle(string ürünAdı, decimal fiyat, int adet = 1)
    {
        _toplam += fiyat * adet;
    }
}
// Test artık geçiyor – YEŞİL

```

### Adım 3 — Yeni Test: İndirim Senaryosu

```

[Fact]
public void İndirimUygula_YüzdeCinsinden_ToplamAzalır()
{
    var sepet = new AlışverişSepeti();
    sepet.Ekle("Defter", fiyat: 20.0m, adet: 1);

    sepet.İndirimUygula(yüzde: 10); // %10 indirim

    Assert.Equal(18.0m, sepet.Toplam); // 20 - 2 = 18
}
// Test BAŞARISIZ – İndirimUygula yöntemi yok

```

### Adım 4 — Yeşil: İndirim Yöntemini Ekle

```

public class AlışverişSepeti
{
    private readonly List<(string Ad, decimal Fiyat, int Adet)> _kalemler = new();

    public decimal Toplam => _kalemler.Sum(k => k.Fiyat * k.Adet) * (1 - _indirimOranı);
    private decimal _indirimOranı = 0;

    public void Ekle(string ürünAdı, decimal fiyat, int adet = 1)
    {
        if (fiyat < 0) throw new ArgumentException("Fiyat negatif olamaz.");
        if (adet <= 0) throw new ArgumentException("Adet pozitif olmalıdır.");

        _kalemler.Add((ürünAdı, fiyat, adet));
    }

    public void İndirimUygula(decimal yüzde)
    {
        if (yüzde < 0 || yüzde > 100)
            throw new ArgumentOutOfRangeException(nameof(yüzde), "0-100 arasında olmalı.");
        _indirimOranı = yüzde / 100;
    }

    public int ÜrünSayısı => _kalemler.Sum(k => k.Adet);
}

```

### Adım 5 — Yeniden Düzenleme ve Kapsamlı Test Suite

```

public class AlışverişSepetTestleri
{
    [Fact]
    public void Ekle_TekÜrün_ToplamDoğru()
    {
        var sepet = new AlışverişSepeti();
        sepet.Ekle("Kalem", 5.0m, 2);
        Assert.Equal(10.0m, sepet.Toplam);
    }
}

```

```

[Fact]
public void Ekle_NegatifFiyat_ArgumentExceptionFırlatır()
{
    var sepet = new AlışverişSepeti();
    Assert.Throws<ArgumentException>(() => sepet.Ekle("Kalem", -1.0m));
}

[Fact]
public void İndirimUygula_YüzdeCinsinden_ToplamAzalır()
{
    var sepet = new AlışverişSepeti();
    sepet.Ekle("Defter", 20.0m);
    sepet.İndirimUygula(10);
    Assert.Equal(18.0m, sepet.Toplam);
}

[Fact]
public void İndirimUygula_Tamİndirim_SıfırToplam()
{
    var sepet = new AlışverişSepeti();
    sepet.Ekle("Ürün", 50.0m);
    sepet.İndirimUygula(100);
    Assert.Equal(0.0m, sepet.Toplam);
}

[Theory]
[InlineData(-1)]
[InlineData(101)]
public void İndirimUygula_GeçersizYüzde_ArgumentOutOfRangeExceptionFırlatır(decimal yüzde)
{
    var sepet = new AlışverişSepeti();
    Assert.Throws<ArgumentOutOfRangeException>(() => sepet.İndirimUygula(yüzde));
}

[Fact]
public void ÜrünSayısı_ÇokKalem_ToplamAdetDöndürür()
{
    var sepet = new AlışverişSepeti();
    sepet.Ekle("Kalem", 2.0m, 3);
    sepet.Ekle("Defter", 5.0m, 2);
    Assert.Equal(5, sepet.ÜrünSayısı);
}
}

```

## 22.7 Test Organizasyonu ve En İyi Uygulamalar

### §22.7.1 Test Projesi Klasör Yapısı

Büyük projelerde test organizasyonu kritik öneme sahiptir. Önerilen klasör yapısı aşağıdaki gibidir;

```

Çözüm/ → Çözüm (Solution)
├── src/ → Klasör (Directory)
│   ├── HesapMakinesi/ → Proje (Console Application)
│   │   ├── HesapMakinesi.csproj → Proje Dosyası (Project)
│   │   ├── HesapMakinesi.cs → Ana (Main) Program
│   │   └── Servisler/ → Klasör (Directory)
│   │       └── KullanıcıServisi.cs → Servis Kodu (Class)
│   ├── Kütüphane.Ortak/ → Klasör (Directory)
│   │   └── Ortak.cs → Proje Kodu (Class)
│   └── tests/ → Klasör (Directory)
│       ├── HesapMakinesi.BirimTestleri/ → Proje (Console Application)
│       └── HesapMakinesi.BirimTestleri.csproj → Proje Dosyası (Project)

```

```

├─ HesapMakinesiTestleri.cs → Proje Kodu (Class)
│   └─ Servisler/ → Klasör (Directory)
│       ├── KullanıcıServisTestleri.cs → Servis Kodu (Class)
│       └─ HesapMakinesi.EntegrasyonTestleri/ → Klasör (Directory)
│           └─ VeritabanıEntegrasyonTestleri.cs → Servis Kodu (Class)
└─ Çözüm.sln → Çözüm Dosyası (Solution)

```

### §22.7.2 Testleri Kategorilere Ayırma

[Trait] Özniteliği ile testler kategorilere ayrılabilir;

```

// Testleri kategorilere ayır
[Trait("Kategori", "BirimTesti")]
[Trait("Modül", "HesapMakinesi")]
public class KategorilenmişTestler
{
    [Fact]
    [Trait("Öncelik", "Kritik")]
    public void Kritik_İş_Kuralı_Doğrulama()
    {
        // Kritik iş kuralı testi
        Assert.True(true);
    }
}

// Yalnızca kritik testleri çalıştır:
// dotnet test --filter "Öncelik=Kritik"
// dotnet test --filter "Kategori=BirimTesti"
// dotnet test --filter "FullyQualifiedNames~HesapMakinesi"

```

### §22.7.3 Temiz Test Kodu İlkeleri

```

// KÖTÜ – Her şey tek yöntemde, anlaşılmaz
[Fact]
public void Test1()
{
    var x = new HesapMakinesi();
    Assert.Equal(7, x.Topla(3, 4));
    Assert.Equal(-1, x.Çıkar(3, 4));
    Assert.Throws<DivideByZeroException>(() => x.Böl(1, 0));
}

// İYİ – Her test tek bir şey doğrular
[Fact]
public void Topla_PozitifSayılar_DogruToplam() { ... }

[Fact]
public void Çıkar_BüyüktenKüçüğü_NegatifSonuç() { ... }

[Fact]
public void Böl_SıfırBölen_İstisnaFırlatır() { ... }

// Test yardımcı yöntemleri ile tekrarı önle
public class SepetTestYardımcısı
{
    // Hazır sepet oluşturucu – DRY (Kendini Tekrar Etme) ilkesi
    public static AlışverişSepeti DoluSepetOluştur(
        decimal toplamTutar = 100.0m,
        string ürünAdı = "Test Ürünü")
    {
        var sepet = new AlışverişSepeti();
        sepet.Ekle(ürünAdı, toplamTutar);
        return sepet;
    }
}

```



```
// Testlerde kullan
[Fact]
public void İndirim_DoluSepet_DoğruHesaplar()
{
    var sepet = SepetTestYardımcısı.DoluSepetOluřtur(100.0m);
    sepet.İndirimUygula(20);
    Assert.Equal(80.0m, sepet.Toplam);
}
```

## §22.7.4 Kod Kapsama Analizi

Testlerin  retim kodunun ne kadarını kapsadıđı **kod kapsama** (**code coverage**) ile  l  l r.

```
// Kod kapsama ile test  alıřtır:
// dotnet test --collect:"XPlat Code Coverage"

// Coverlet ile daha ayrıntılı rapor:
// dotnet add package coverlet.collector
// dotnet test /p:CollectCoverage=true /p:CoverletOutputFormat=lcov

// coverlet  ıktısı:
// +-----+-----+-----+-----+
// | Module           | Line   | Branch | Method |
// +-----+-----+-----+-----+
// | HesapMakinesi.dll | 95.2%  | 87.5%  | 100%   |
// +-----+-----+-----+-----+
```

**Uyarı:** %100 kod kapsama hedefi, dođruluk garantisi vermez.  nemli olan anlamlı senaryoların (mutlu yol, hata yolları, sınır deđerleri) test edilmesidir. Kod kapsamayı bir metrik olarak kullanın, ama  olarak deđil.

## 22.8 D ř n ve   z

-   **D ř n:** Test G d ml  Geliřtirme'nin klasik " nce kod sonra test" yaklařımına g re avantajları ve dezavantajları nelerdir? Bir yazılımcı TDD'yi her zaman uygulamalı mıdır? Hangi durumlarda TDD uygulamak anlamsız olabilir?

-   **D ř n:** Ařađıdaki kodda test edilmesi zor olan nedir? Nasıl yeniden tasarlanmalıdır?

```
public class RaporServisi
{
    public string RaporOluřtur()
    {
        // DateTime.Now ve new HttpClient dođrudan kullanılıyor
        var tarih = DateTime.Now;
        using var istemci = new HttpClient();
        var veri = istemci.GetStringAsync("https://api.example.com").Result;
        return $"[{tarih:dd.MM.yyyy}] {veri}";
    }
}
```

-   **  z:** Bir banka hesabı sınıfı i in TDD ile geliřtirme yapın. Ařađıdaki gereksinimleri adım adım (Kırmızı→Yeřil→Yeniden D zenle) uygulayın:

-   Hesap a ılıřta bařlangı  bakiyesiyle oluřturulabilir
-   Para yatırılabilir (negatif tutar kabul edilmez)
-   Para  ekilebilir (bakiye yetersizse hata fırlatır)
-   Bakiye sorgulanabilir
-   İřlem ge miři tutulur

-   **  z:** *İHavaServisi* aray z n  uygulayan bir *HavaDurumuServisi* sınıfı yazınız. Bu sınıfı kullanan sınıfta *G nl kPlanlamacı.Plan ner()* y ntemini geliřtirin. Mo  ile *İHavaServisi*'ni taklit ederek d rt farklı hava durumu senaryosunu test edin (yađmurlu, g neřli, karlı, fırtınalı).

-   **D ř n:** Mock ve stub arasındaki fark nedir? Mo 'da *Setup().Returns()* ile stub, *Verify()* ile mock davranıřı kurulur. Bir testte hem stub hem mock kullanmak gerekiyorsa bu testin  ok řeyi dođrulamaya  alıřtıđının iřareti midir?

-   **  z:** *IDepoServisi* aray z n  hem eř zamanlı hem eř zamansız y ntemlerle tasarlayın. xUnit ve Mo  kullanarak bu servisi kullanan *async Task< r n[]> Aktif r nleriGetirAsync()* y ntemini test edin.

Eş zamansız istisna senaryolarını da kapsayın.

- ◇ **Düşün:** %80 kod kapsama hedefi anlamlı mıdır? Kapsama oranı yüksek ancak anlamsız testler mi daha iyidir, yoksa oranı düşük ama kritik senaryoları kapsayan testler mi? Bir projede test kalitesini nasıl ölçersiniz?